LAPORAN PRAKTIKUM STRUKTUR DATA

Queue

Disusun Oleh :

Aliifah Felda Mufarrihati Salwaa

2511531011

Dosen Pengampu :

Dr. Wahyudi, M.T

Asisten Praktikum :

M Galid Avaro

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Laporan praktikum ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum Struktur Data yang membahas tentang Queue. Melalui laporan ini penulis dapat memahami materi praktikum secara mendalam dan dapat lebih teliti, teratur, serta memiliki kemampuan yang baik dalam penulisan kode program sesuai kaidah akademik. Sehingga laporan ini dapat menjadi sarana belajar, dokumentasi kegiatan, dan referensi praktikum berikutnya.

Penulis menyadari bahwa laporan ini masih memiliki banyak kekurangan, baik dari isi maupun penyajiannya. Oleh karena itu, saran dan kritik sangat penulis harapkan guna untuk menyempurnakan laporan berikutnya.

Padang, 28 April 2026

Aliifah Felda Mufarrihati Salwaa
2511531011

DAFTAR ISI

KATA PENGANTAR i

BAB I PENDAHULUAN 3

1.1 Latar Belakang Praktikum 3

1.2 Tujuan Praktikum 3

1.3 Manfaat Praktikum 3

BAB II PEMBAHASAN 4

2.1. Dasar Teori 4

2.2. Class QueueArray_2511531011 5

2.3. QueueArrayDriver_2511531011 6

2.4. ReverseData_2511531011 7

2.5. IterasiQueue_2511531011 8

2.6. QueueLinkedList_2511531011 9

2.7. Tugas Pekan 4 10

BAB III KESIMPULAN 15

3.1. Kesimpulan 15

3.2. Saran 15

DAFTAR PUSTAKA 16

BAB I
PENDAHULUAN

1.1 Latar Belakang Praktikum

Setelah mempelajari salah satu implementasi ADT yakni array list dan stack pada praktikum sebelumnya, kita juga harus mengetahui tentang penerapan ADT yang lain. Karena banyaknya implementasi ADT, kita harus dapat menguasai setiap implementasinya agar dapat membangun sebuah program efektif dan efisien, namun tetap sesuai dengan keinginan kita.

Oleh karena itu, pada praktikum kali ini kita akan mempelajari dan memahami tentang penerapan ADT yang lainnya yakni Queue. Queue menggunakan prinsip FIFO (*First In First Out*) yang berarti elemen pertama yang dimasukkan ke dalam antrean menjadi yang pertama pula keluar dari antrean.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut:

1. Memahami tentang implementasi ADT yakni Queue.
2. Mengetahui kelebihan dan kekurangan Queue dalam program.
3. Dapat membangun sebuah kode program dengan Queue.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini adalah sebagai berikut:

1. Mampu mengimplementasikan Queue dalam pembuatan program agar lebih efektif dan efisien.
2. Memahami penggunaan Queue dalam program.

BAB II
PEMBAHASAN

2.1. Dasar Teori

Queue atau antrian adalah struktur data yang menyusun elemen-elemen data dalam urutan linier. Prinsip yang digunakan pada queue adalah FIFO (*First In First Out*) yang berarti elemen data yang pertama dimasukkan ke dalam antrean akan menjadi yang pertama pula untuk dikeluarkan. Ujung antrean atau bagian depan antrean disebut sebagai *front* atau *head*. Bagian belakang tempat menambahkan elemen disebut dengan *rear* atau *tail*.

Operasi dasar queue:

1. Enqueue: Proses menambahkan sebuah elemen ke dalam antrean pada posisi *rear*. Jika antrean penuh, operasi ini akan menghasilkan kondisi *overflow*.
2. Dequeue: Proses mengeluarkan elemen dari antrian pada posisi *front*. Jika antrean kosong, operasi ini akan menghasilkan kondisi *underflow*.

Queue juga memiliki beberapa kelebihan dan kekurangan, yakni:

A. Kelebihan
    1. Keteraturan data.
    2. Decoupling Sistem: produsen dan konsumen data tidak perlu berjalan pada waktu yang sama.
    3. Pengelolaan Aliran Data: bertindak sebagai *buffer* yang menstabilkan sistem
    4. Efisiensi Operasi Dasar
B. Kekurangan
    1. Akses Terbatas
    2. Konsumsi memori
    3. Tidak selalu cocok di setiap skenario

2.2. Class QueueArray_2511531011

```
package pekan4_2511531011;

public class QueueArray_2511531011 {
	int front_1011, rear_1011, size_1011;
	int capacity_1011;
	int array_1011[];

	public QueueArray_2511531011(int capacity_1011) {
		this.capacity_1011 = capacity_1011;
		front_1011 = this.size_1011;
		rear_1011 = capacity_1011 - 1;
		array_1011 = new int [this.capacity_1011];
	}

	boolean isFull_2511531011 (QueueArray_2511531011 queue_1011) {
		return (queue_1011.size_1011 == queue_1011.capacity_1011);
	}

	boolean isEmpty_2511531011 (QueueArray_2511531011 queue_1011) {
		return (queue_1011.size_1011 == 0);
	}

	void enqueue_2511531011 (int item_1011) {
		if (isFull_2511531011(this))
			return;
		this.rear_1011 = (this.rear_1011 + 1) % this.capacity_1011;
		this.array_1011[this.rear_1011] = item_1011;
		this.size_1011 = this.size_1011 + 1;
		System.out.println(item_1011 + " enqueued to queue");
	}

	int dequeue_2511531011() {
		if (isEmpty_2511531011(this))
			return Integer.MIN_VALUE;
		int item_1011 = this.array_1011[this.front_1011];
		this.front_1011 = (this.front_1011 + 1) %
this.capacity_1011;
		this.size_1011 = this.size_1011 - 1;
		return item_1011;
	}

	int front_2511531011() {
		if (isEmpty_2511531011(this))
			return Integer.MIN_VALUE;

		return this.array_1011[this.front_1011];
	}

	int rear_2511531011() {
		if (isEmpty_2511531011(this))
			return Integer.MIN_VALUE;
		return this.array_1011[this.rear_1011];
	}
```

```
            //mencetak elemen antrian
        void display_2511531011() {
            int i;
            if (front_1011 == rear_1011) {
                System.out.printf("\nAntrian Kosong\n");
                return;
            }
            //kunjungi dari belakang dan cetak
            for (i = front_1011; i < rear_1011; i++) {
                System.out.printf(" %d <-- ", array_1011[i]);
            }
            return;
        }

}
```

Kode program 2.1: QueryArray_2511531011

Class ini merupakan ADT dan konstruktor yang akan digunakan pada class queueArrayDriver. Metode yang dibuat pada class ini adalah:

1. QueueArray_2511531011: sebagai konsturktor array
2. isFull_2511531011: untuk melihat apakah array sudah full
3. isEmpty_2511531011: memastikan array tidak kosong
4. enqueue_2511531011: memasukkan nilai kedalam array
5. dequeue_2511531011: mengeluarkan nilai dari array
6. front_2511531011: melihat nilai paling depan
7. rear_2511531011: melihat nilai paling belakang
8. display_2511531011: mencetak semua elemen yang ada pada array

2.3. QueueArrayDriver_2511531011

```
package pekan4_2511531011;

public class QueueArrayDriver_2511531011 {

        public static void main(String[] args) {
            QueueArray_2511531011 queue_1011 = new
QueueArray_2511531011(1000);
            queue_1011.enqueue_2511531011(10);
            queue_1011.enqueue_2511531011(20);
            queue_1011.enqueue_2511531011(30);
            queue_1011.enqueue_2511531011(40);
            System.out.println("Item di depan " +
queue_1011.front_2511531011());
            System.out.println("Item paling belakang " +
queue_1011.rear_2511531011());
```

```
            System.out.println("Tampilan queue");
            queue_1011.display_2511531011();
            System.out.println();
            System.out.println(queue_1011.dequeue_2511531011() + "
dihapus dari queue");
            System.out.println("Item di depan: " +
queue_1011.front_2511531011());
            System.out.println("Item paling belakang " +
queue_1011.rear_2511531011());
            System.out.println("tampilan queue setelah satu data
dihapus");
            queue_1011.display_2511531011();
      }
}
```

Kode program 2.2: QueueArrayDriver_2511531011

Pada kode program ini kita akan memasukkan beberapa nilai ke dalam array dengan metode enqueue yang telah disediakan pada class QueueArray sebelumnya. Kita disini juga menggunakan metode untuk melihat nilai awal, melihat nilai akhir, dan mengeluarkan sebuah nilai. Hingga pada akhirnya kita dapat mencoba sendiri contoh penggunaan queue dari tampilan awal hingga tampilan akhir setelah dimodifikasi.

Output :

```
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item paling belakang 40
Tampilan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
Item di depan: 20
Item paling belakang 40
tampilan queue setelah satu data dihapus
 20 <--  30 <--
```

2.4. ReverseData_2511531011

```
package pekan4_2511531011;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511531011 {
```

```java
	public static void main(String[] args) {
		Queue<Integer> q_1011 = new LinkedList<Integer>();
		q_1011.add(1);
		q_1011.add(2);
		q_1011.add(3); // [1, 2, 3]
		System.out.println("sebelum reverse: " + q_1011);
		Stack<Integer> s_1011 = new Stack<Integer>();
		while (!q_1011.isEmpty()) { // Q -> S
			s_1011.push(q_1011.remove());
		}
		while (!s_1011.isEmpty()) { // S -> Q
			q_1011.add(s_1011.pop());
		}
		System.out.println("sesudah reverse: " + q_1011); // [3, 2,
1]

	}

}
```

Kode program 2.3: ReverseData_2511531011

Pada program ini kita akan melakukan kombinasi antara queue dengan stack untuk mengubah urutan sebuah elemen. Kita akan menjadikan q_1011 sebagai queue dan diisi dengan nilai 1, 2, 3. Kemudian kita membuat sebuah stack dengan variabel s_1011. Kita akan menjalan kondisi dimana jika queue tidak kosong, maka kita akan mengisi nilai ke dalam stack dengan nilai *remove* q (nilai yang dikeluarkan dari q. Selain itu, kita juga akan melihat kondisi dimana jika s tidak kosong, maka kita akan mengisi q dengan nilai s yang di pop. Pada akhirnya, nilai q yang awalnya [1, 2, 3] akan berubah jadi [3, 2, 1] karena adanya pertukaran letak data.

Output :

```
sebelum reverse: [1, 2, 3]
sesudah reverse: [3, 2, 1]
```

2.5. IterasiQueue_2511531011

```java
package pekan4_2511531011;

import java.util.Iterator;
import java.util.Queue;
import java.util.LinkedList;

public class IterasiQueue_2511531011 {

	public static void main(String[] args) {
```

```
            Queue<String> q_1011 = new LinkedList<>();
            q_1011.add("Praktikum");
            q_1011.add("Struktur");
            q_1011.add("Data");
            q_1011.add("Dan");
            q_1011.add("Algoritma");
            Iterator<String> iterator_1011 = q_1011.iterator();
            while (iterator_1011.hasNext()) {
                  System.out.print(iterator_1011.next() + " ");
            }

      }

}
```

Kode prorgam 2.4: IterasiQueue_2511531011

Kode program ini sejatinya merupakan implementasi queue paling sederhana, yakni memasukkan data dengan add dan kemudian mengecek apakah seriap iterasi memiliiki lanjutan. Jika ya, maka kita akan print semua iterasi yang ada dan jika false, kita akan berhenti.

Output :

```
<terminated> IterasiQueue_2511531011 [Java Applicatio
Praktikum Struktur Data Dan Algoritma
```

2.6. QueueLinkedList_2511531011

```
package pekan4_2511531011;

import java.util.Queue;
import java.util.LinkedList;

public class QueueLinkedList_2511531011 {

      public static void main(String[] args) {
            Queue<Integer> q_1011 = new LinkedList<>();
            // Tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
            for (int i = 0; i < 6; i++)
                  q_1011.add(i);
            // Menampilkan isi antrian
            System.out.println("Elemen Antrian " + q_1011);
            // Untuk menghapus kepala antrian
```

```
            int hapus_1011 = q_1011.remove();
            System.out.println("Hapus elemen = " + hapus_1011);
            System.out.println(q_1011);
            // Untuk melihat antrian terdepan
            int depan_1011 = q_1011.peek();
            System.out.println("Kepala Antrian = " + depan_1011);

            int banyak_1011 = q_1011.size();
            System.out.println("Size antrian = " + banyak_1011);
        }

}
```

Kode program 2.5: QueueLinkedList_2511531011

Pada program ini kita akan melakukan queue biasa yang menggunakan LinkedList. Awalnya kita akan membuat konstruktor dengan melakukan iterasi untuk memasukkan 6 elemen ke dalam sebuah array q_1011. Kemudian kita hanya melakukan proses sederhana seperti menampilkan semua isi array, menghapus kepala antrian dengan remove, melihat antrian terdepan dengan peek, dan melihat banyaknya size antrian.

Output :

```
<terminated> QueueLinkedList_2511531011 [Java Ap
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size antrian = 5
```

2.7. Tugas Pekan 4

A. AntrianLoket_2511531011

```
package pekan4_2511531011;

import java.util.*;

public class AntrianLoket_2511531011 {
        int front_1011, rear_1011, size_1011;
        int capacity_1011;
        String array_1011[];

        public AntrianLoket_2511531011(int capacity_1011) {
            this.capacity_1011 = capacity_1011;
```

```
            front_1011 = this.size_1011;
            rear_1011 = capacity_1011 - 1;
            array_1011 = new String [this.capacity_1011];
        }

        boolean isFull_2511531011 (AntrianLoket_2511531011 queue_1011) {
            return (queue_1011.size_1011 == queue_1011.capacity_1011);
        }

        boolean isEmpty_2511531011 (AntrianLoket_2511531011 queue_1011) {
            return (queue_1011.size_1011 == 0);
        }

        void enqueue_2511531011 (String item_1011) {
            if (isFull_2511531011(this))
                return;
            this.rear_1011 = (this.rear_1011 + 1) % this.capacity_1011;
            this.array_1011[this.rear_1011] = item_1011;
            this.size_1011 = this.size_1011 + 1;
            System.out.println(item_1011 + " enqueued to queue");
        }

        String dequeue_2511531011() {
            if (isEmpty_2511531011(this))
                return null;
            String item_1011 = this.array_1011[this.front_1011];
            this.front_1011 = (this.front_1011 + 1) %
this.capacity_1011;
            this.size_1011 = this.size_1011 - 1;
            return item_1011;
        }

            //mencetak elemen antrian
        void display_2511531011() {
            int i;
            if (front_1011 == rear_1011) {
                System.out.printf("\nAntrian Kosong\n");
                return;
            }
            //kunjungi dari belakang dan cetak
            for (i = front_1011; i < rear_1011; i++) {
                System.out.printf(" %d <-- ", array_1011[i]);
            }
            return;
        }

        public void reverse_2511531011() {
            if (isEmpty_2511531011(this)) {
                System.out.println("Antrian kosong");
                return;
                }
            Stack<String> stack = new Stack<>();
            int n = size_1011;

            // Queue → Stack
            for (int i = 0; i < n; i++) {
                stack.push(dequeue_2511531011());
```

```
                }

            // Stack → Queue
            while (!stack.isEmpty()) {
                enqueue_2511531011(stack.pop());
                }

            System.out.println("Antrian berhasil dibalik");
    }

}
```

B. AntrianLoketDriver_2511531011

```
package pekan4_2511531011;

import java.util.*;

public class AntrianLoketDriver_2511531011 {
            public static void tampilkanMenu_2511531011() {
                System.out.println("\n1. Tambah Antrian");
                System.out.println("2. Hapus Antrian");
                System.out.println("3. Tampilkan Antrian");
                System.out.println("4. Reverse");
                System.out.println("5. Keluar");
            }

            public static void main (String[] args) {
                Scanner scanner = new Scanner(System.in);
                AntrianLoket_2511531011 antrian = new
AntrianLoket_2511531011(10);
                int choice;

                System.out.println("=== Program Antrian Loket ===");

                do {
                    tampilkanMenu_2511531011();
                    System.out.print("\nPrint Menu: ");
                    choice = scanner.nextInt();
                    scanner.nextLine();

                    switch (choice) {
                    case 1:
                        System.out.print("Masukkan nama
pelanggan:");
                        String nama_1011 = scanner.nextLine();
                        antrian.enqueue_2511531011(nama_1011);
                        break;
                    case 2:
                        String keluar =
antrian.dequeue_2511531011();
                        if (keluar == null) {
                            System.out.println("Antrian
kosong");
                            } else {
```

```
                                                System.out.println(keluar
+ " telah dilayani");
                                                }
                                    break;
                                case 3:
                                    antrian.display_2511531011();
                                    break;
                                case 4:
                                    antrian.reverse_2511531011();
                                    break;
                                case 5:
                                    System.out.println("Program Selesai");
                                    break;
                                default:
                                    System.out.println("Pilihan tidak
valid");
                                }
                            } while (choice !=5);
                            scanner.close();
                        }

}
```

Output:

```
=== Program Antrian Loket ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Print Menu: 1
Masukkan nama pelanggan:budi
budi enqueued to queue

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Print Menu: 2
budi telah dilayani

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Print Menu: 3

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Print Menu: 4
Antrian kosong
```

```
Print Menu: 4
Antrian kosong

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Print Menu: 5
Program Selesai
```

BAB III
KESIMPULAN

3.1. Kesimpulan

Penggunaan stack memiliki keuntungan sendiri, dimana kita bisa mengoperasikan data teratas tanpa mengganggu data yang paling bawah. Namun, penggunaan stack juga harus diperhatikan. Jika menggunakan stack pada suatu data yang terus berubah, maka kita akan mengalami kesulitan nyata. Namun, jika implementasi stack digunakan dengan baik, kita akan menghasilkan program yang lebih efektif dan efisien.

3.2. Saran

Karena banyaknya implementasi dari ADT, maka mahasiswa sebaiknya terus mengulang dan mempelajari setiap implementasi ADT. Mahasiswa kini hendaknya sudah mampu untuk membedakan antara array list dengan stcak. Untuk lebih mengasah keterampilan demi mendapatkan hasil program yang baik, hendaknya mahasiswa mampu dan terus belajar, salah satunya implementasi ADT.

DAFTAR PUSTAKA

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, 3rd ed., Cambridge, MA: MIT Press, 2009.

[2] Wahyudi, *Struktur Data dan Algoritma,* Padang: Informatika, 2026

[3] M.T. Goodrich, *Data Structures and Algorithms in Java*